

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.
2. To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

```
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir src
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir include
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir obj
mkdirobj: command not found
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir bin
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir docs
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir tests
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir scripts
ramya@LAPTOP-OPDAF309:~/skill1$ mkdir assets
ramya@LAPTOP-OPDAF309:~/skill1$ touch .gitignore
ramya@LAPTOP-OPDAF309:~/skill1$ touch README.md
ramya@LAPTOP-OPDAF309:~/skill1$ touch LICENSE
ramya@LAPTOP-OPDAF309:~/skill1$ touch Mkaefile
ramya@LAPTOP-OPDAF309:~/skill1$ touch src/main.c
ramya@LAPTOP-OPDAF309:~/skill1$ touch src/shell.c
ramya@LAPTOP-OPDAF309:~/skill1$ touch src/parser.c
ramya@LAPTOP-OPDAF309:~/skill1$ touch src/executor.c
ramya@LAPTOP-OPDAF309:~/skill1$ touch src/builtin.c
ramya@LAPTOP-OPDAF309:~/skill1$ touch include/shell.h
ramya@LAPTOP-OPDAF309:~/skill1$ touch include/parser.h
ramya@LAPTOP-OPDAF309:~/skill1$ touch include/executor.h
ramya@LAPTOP-OPDAF309:~/skill1$ touch include/builtin.h
ramya@LAPTOP-OPDAF309:~/skill1$ touch docs/design.md
ramya@LAPTOP-OPDAF309:~/skill1$ touch docs/report.pdf
ramya@LAPTOP-OPDAF309:~/skill1$ touch tests/test_parser.c
ramya@LAPTOP-OPDAF309:~/skill1$ touch tests/test_executor.c
ramya@LAPTOP-OPDAF309:~/skill1$ touch scripts/run.sh
ramya@LAPTOP-OPDAF309:~/skill1$ touch assets/architecture.png
```

```
ramya@LAPTOP-OPDAF309:~/skill1$ git init
hint: Using 'master' as the name for the initial branch. This default branch
hint: name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warni
hint: ng,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command
hint: :
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName fal
hint: se"
Initialized empty Git repository in /home/ramya/skill1/.git/
```

```
ramya@LAPTOP-OPDAF309:~/skill1$ find . -maxdepth 3 -print
```

```
.  
./git  
./git/refs  
./git/refs/heads  
./git/refs/tags  
./git/objects  
./git/objects/pack  
./git/objects/info  
./git/config  
./git/description  
./git/info  
./git/info/exclude  
./git/HEAD  
./git/hooks  
./git/hooks/pre-rebase.sample  
./git/hooks/post-update.sample  
./git/hooks/pre-applypatch.sample  
./git/hooks/commit-msg.sample  
./git/hooks/pre-receive.sample  
./git/hooks/pre-push.sample  
./git/hooks/pre-commit.sample  
./git/hooks/applypatch-msg.sample  
./git/hooks/sendemail-validate.sample  
./git/hooks/pre-merge-commit.sample  
./git/hooks/prepare-commit-msg.sample  
./git/hooks/push-to-checkout.sample  
./git/hooks/update.sample  
./git/hooks/fsmonitor-watchman.sample  
./bin  
./src  
./src/main.c  
./src/builtin.c  
./src/parser.c  
./src/executor.c  
./src/shell.c  
./scripts  
./scripts/run.sh  
./gitignore  
./assets
```

```
./assets/architecture.png
./LICENSE
./README.md
./tests
./tests/test_executor.c
./tests/test_parser.c
./include
./include/parser.h
./include/shell.h
./include/builtin.h
./include/executor.h
./Mkaefile
./obj
./docs
./docs/design.md
./docs/report.pdf
ramya@LAPTOP-OPDAF309:~/skill1$ man 2 fork
ramya@LAPTOP-OPDAF309:~/skill1$ man 3 exec
ramya@LAPTOP-OPDAF309:~/skill1$ man 2 execve
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main(void)
{
    char command[100];

    while (1)
    {
        printf("my_shell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
        {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        if (strlen(command) == 0)
        {
            continue;
        }

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        pid_t pid = fork();

        if (pid < 0)
```

```
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());

        execlp(command, command, (char *)NULL);

        perror("exec failed");
        exit(1);
    }
    else
    {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        waitpid(pid, NULL, 0);
    }
}

return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill1$ nano src/main.c
ramya@LAPTOP-OPDAF309:~/skill1$ gcc src/main.c -o bin/my_shell
ramya@LAPTOP-OPDAF309:~/skill1$ ls -l bin/my_shell
-rwxr-xr-x 1 ramya ramya 16560 Aug 26 04:56 bin/my_shell
ramya@LAPTOP-OPDAF309:~/skill1$ bin/my_shell
my_shell>
my_shell>
my_shell>
my_shell> ls
This is the Child Process.
This is the Parent Process.
Parent PID = 1148
Child PID = 1152
Child PID = 1152
LICENSE  README.md  bin    include  scripts  tests
Mkaefile assets    docs  obj      src
my_shell> pwd
This is the Parent Process.
Parent PID = 1148
Child PID = 1153
This is the Child Process.
Child PID = 1153
/home/ramya/skill1
my_shell> date
This is the Parent Process.
Parent PID = 1148
Child PID = 1154
This is the Child Process.
Child PID = 1154
Wed Aug 26 04:57:20 UTC 2026
my_shell> whoami
This is the Parent Process.
Parent PID = 1148
Child PID = 1156
This is the Child Process.
Child PID = 1156
ramya
my_shell> exit
Exiting shell...
```

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
    }
    else if (pid == 0)
    {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        wait(NULL);
    }

    return 0;
}

```

```

ramya@LAPTOP-OPDAF309:~/skill1$ nano src/fork_test.c
ramya@LAPTOP-OPDAF309:~/skill1$ gcc src/fork_test.c -o bin/fork_test
ramya@LAPTOP-OPDAF309:~/skill1$ ./bin/fork_test
This is the Parent Process.
Parent PID = 1205
Child PID = 1206
This is the Child Process.
Child PID = 1206

```